Project 3: Chosen Data Structure

- How expensive is it to pull out the minimum node with each approach? How expensive is it to add a newly-combined internal node back in?
- Does the cost of building the initial collection of leaf nodes differ between the two approaches?
- Roughly how many times will you remove-the-minimum and add-a-node over the course of building a tree with ‘n’ unique characters? How does that interact with the per-operation cost of each data structure?
- If two nodes have equal occurrence counts, does your chosen data structure make it easy or awkward to break the tie the way comesBefore specified?

1. With an ordered ArrayList, the minimum is always at the front of the list, so removing it costs O(1). With a min-heap priority queue, remoivng the minimum is O(log n). Inserting to a correct sorted location is O(n) because elements may need to be shifted. In a min-heap, insertion is O(log n).
2. Adding n leaf nodes one at a time to an ordered list is O(n^2) overall because each insertion can take up to o(n). Adding them to a priority queue is O(n log n) overall. A heap can also be built from all nodes at once in O(n.
3. For n unique characters, the algorithm combines two nodes until one tree remains,
    a. 2(n-1) remove-min operations
    b. n - 1 add-node operations

With an ordered list, each add is O(n), while remove-min is O(1). With a priority queue, both are O(log n). The priority queue is more efficient for large inputs, but there are at most 256 possible ASCII characters here, so an ordered list is still fast enough.

Decision

I chose an ordered ArrayList<HuffmanNode> to hold nodes that have not yet been combined into the final Huffman tree. addNode inserts each node at the first position where it comes before the existing node, keeping the list sorted from smallest to largest.

Cost of remove-min and add-node

With an ordered list, removing the minimum is O(1) because it is at the front. Adding a node is O(n), since elements after the insertion point may shift. A min-heap priority queue would make both insertion and remove-min O(log n).

Cost of the initial leaf collection

Repeated ordered-list insertion costs $O(n^2)$ to build the initial collection. Adding the leaves to a priority queue costs O(n log n), or O(n) if a heap is built all at once.

Work while building the tree

For n unique characters, tree construction performs 2(n − 1) remove-min operations and n − 1 addition. Even though a heap has better asymptotic performance, this project has at most 256 distinct 8-bit characters. The ordered list is therefore small, easy to inspect, and fully adequate.

Tie-breaking

Tie-breaking is direct and reliable with the ordered list: every insertion uses comesBefore, which compares frequency first and then the smallest ASCII value. removeMin also uses this same rule for any Collection, ensuring that the tree remains deterministic.